

Unit 2 Assignment (Attempt any 5)

Complexity & Notations

1. Define Big-Theta (Θ) notation with a suitable example.
2. Differentiate between Big-O, Big- Ω , and Big- Θ notations.
3. Explain space complexity and its components with an example.
4. Compare time complexity and space complexity of algorithms.

Searching & Sorting Basics

5. Why do we need searching and sorting techniques? Explain with real-life examples.
6. Differentiate between internal sorting and external sorting.
7. What is sort stability? Give examples of stable and unstable sorting algorithms.
8. Write a short note on the importance of sorting before applying Binary Search.

Searching Algorithms

9. Explain the working of **Linear Search** with an example. Write its complexity.
10. Explain the working of **Binary Search** with an example. Write its complexity.
11. Compare Linear Search and Binary Search in terms of performance and applicability.

Sorting Algorithms

12. Explain the working of **Bubble Sort** with a small example.
13. Write and explain the time and space complexity of **Insertion Sort**.
14. Why is Selection Sort considered inefficient compared to Insertion Sort?
15. Explain the partitioning process in **Quick Sort** with an example.
16. Explain the **Merge procedure** in Merge Sort with an example.
17. Write the properties of Heap Sort and explain why it is not stable.

Comparisons & Analysis

18. Compare Quick Sort and Merge Sort in terms of time, space, and stability.
 19. Write differences between stable and unstable sorting algorithms with examples.
 20. Write a comparison table of all sorting algorithms (Bubble, Insertion, Selection, Quick, Merge, Heap) based on best, worst, and average time complexities.
-

Some Problem statements regarding sorting

Bubble Sort

1. Sort the following list using **Bubble Sort** and show the array after each pass:
35, 20, 15, 40, 50
-

Insertion Sort

2. Sort the following numbers using **Insertion Sort** and show the intermediate steps:
27, 12, 25, 8, 19
 3. Arrange the following elements in ascending order using **Insertion Sort**:
45, 22, 11, 90, 67, 32
-

Selection Sort

4. Apply **Selection Sort** on the following array and show all passes:
29, 10, 14, 37, 14
 5. Sort the numbers using **Selection Sort**:
64, 25, 12, 22, 11
-

Quick Sort

6. Sort the following elements using **Quick Sort**. Take the first element as pivot:
15, 9, 7, 13, 20, 25, 30
 7. Sort the array using **Quick Sort** with the middle element as pivot:
40, 10, 80, 90, 30, 70, 60, 20, 50
-

Merge Sort

8. Sort the following elements using **Merge Sort**. Show the divide and merge steps:
38, 27, 43, 3, 9, 82, 10
 9. Apply **Merge Sort** on:
50, 20, 60, 10, 40, 30
-

Heap Sort

- Sort the following elements using **Heap Sort**. Show max-heap construction and sorting steps:
40, 10, 30, 50, 60, 15
-

Some Problem statements regarding searching

Linear Search Problems

- Use **Linear Search** to find the element 25 in the list:
10, 15, 20, 25, 30, 35
Show the number of comparisons made.
 - Apply **Linear Search** to check whether 50 is present in:
5, 10, 15, 20, 25, 30, 35
 - Using **Linear Search**, find the position of the element 67 in:
45, 67, 89, 12, 90, 32
 - How many comparisons are required in the **worst case** of Linear Search for an array of size $n = 12$?
 - Write the steps of **Linear Search** for key 18 in:
2, 4, 6, 8, 10, 12, 14, 16, 18, 20.
-

Binary Search Problems

(All arrays are already sorted)

- Use **Binary Search** to find element 55 in:
10, 20, 30, 40, 50, 55, 60, 70, 80.
Show all mid-value checks.
- Apply **Binary Search** to find the element 100 in:
10, 25, 40, 55, 70, 85, 100, 115, 130.
- Show step-by-step **Binary Search** for element 23 in:
2, 5, 8, 12, 16, 23, 38, 56, 72.
- How many comparisons are required in the **worst case** of Binary Search for an array of size $n = 64$?
- A sorted list of roll numbers is given:
101, 103, 105, 107, 109, 111, 113.
Use **Binary Search** to find whether roll number 107 is present. Show steps.